Assignment 1
Queries and Functions on CheeseDB

Last updated: **Tuesday 4th March 8:42am**
Most recent changes are shown in red ... older changes are shown in brown.
[Assignment Spec] [Database Design] [Examples] [Testing]

Aims

This assignment aims to give you practice in

- reading and understanding a small relational schema (Cheese DB)
- implementing SQL queries and views to satisfy requests for information
- implementing PLpgSQL functions to aid in satisfying requests for information

The goal is to build some useful data access operations on the Cheese database, which contains information about cheeses.
One aim of this assignment is to use SQL queries (packaged as views) to extract such information. Another is to build PLpgSQL functions that can support higher-level activities, such as might be needed in a Web interface.

Summary

Submission: Login to Course Web Site > Assignments > Assignment 1 > [Submit] > upload `ass1.sql`
or,
on a CSE server, `give cs3311 ass1 ass1.sql`

Required Files: `ass1.sql` (contains both SQL views and PLpgSQL functions)

Deadline: 23:59:59 Friday 21st March 2025

Marks: **13 marks** toward your total mark for this course

Late Penalty: 0.2 *percent* off the achieved mark for each hour late, for 5 days
any submission after 5 days scores 0 marks ... UNSW late penalty policy

How to do this assignment:

- read this specification carefully and completely
- create a directory for this assignment
- copy the supplied files into this directory (see Setting Up)
- login to `vxdb02` and run your PostgreSQL server**
  (or run a PostgreSQL server installed on your home machine)
- load the database and start exploring
- complete the tasks below by editing `ass1.sql`
- test your work on `vxdb02`, which is where it will be marked
- submit `ass1.sql` via WebCMS or `give`

Details of the above steps are given below. Note that you can put the files wherever you like; they do not have to be under your `/localstorage` directory. You should edit your SQL files on hosts other than `vxdb02`. The only time that you need to use `vxdb02` is to manipulate your database. Since you can work on your home machine, you don't have to use `vxdb02` at all while *developing* your solution, but you should definitely test it there before submitting.

Background

According to Wikipedia, *cheese* is "a type of dairy product produced in a range of flavors, textures, and forms by coagulation of the milk protein casein". According to me, cheese is delicious.

Photo by Daderot. Cheese display in Cambridge, MA.

Introduction

In order to work with a database, it is essential to have some background in the domain of data being stored. This database has information about cheeses, cheese makers, cheese styles and places. Cheeses have (trade) names, are made to a style, are made by a particular cheese maker, are aged for a certain amount of time and have a final fat content. Cheese makers are located in one particular location (for our purposes), and were founded in a certain year. Cheese styles have a name, have their origin in a certain location, have a certain hardness (soft, semi-soft, semi-hard, hard), are made from the milk of a certain animal (typically a cow), and are aged for a certain length of time. Locations may be as precise as a particular town in a particular region of a country, or as imprecise as just the country.

The above information is described in more detail both as an ER model and an SQL schema in the schema page.

Doing this Assignment

The following sections describe how to carry out this assignment. Some of the instructions must be followed exactly; others require you to exercise some discretion. The instructions are targetted at people doing the assignment on `vxdb02`. If you plan to work on this assignment at home on your own computer, you'll need to adapt the instructions to "local conditions".

If you're doing your assignment on the CSE machines, some commands must be carried out on `vxdb02`, while others can (and probably should) be done on a CSE machine other than `vxdb02`. In the examples below, we'll use `vxdb02$` to indicate that the comand must be done on `vxdb02` and `cse$` to indicate that it can be done elsewhere.

Setting Up

The first step in setting up this assignment is to set up a directory to hold your files for this assignment.

```
cse$ mkdir /my/dir/for/ass1
cse$ cd /my/dir/for/ass1
cse$ cp /home/cs3311/web/25T1/assignments/ass1/ass1.sql ass1.sql
cse$ ln -s /home/cs3311/web/25T1/assignments/ass1/ass1.dump ass1.dump
```

This gives you a template for the SQL views and SQL and PLpgSQL functions that you need to submit. You edit this file, (re)load the definitions into the database you created for the assignment, and test it there.

As supplied, the `ass1.sql` template file consists entirely of comments; remove the comments from whatever question you're working on. This allows you to load the `ass1.sql` file without getting syntax errors from the not-yet-completed parts of the template. Of course, don't forget to remove the comments from all completed questions before testing and submitting.

The next step is to set up your database:

```
... login to vxdb02, source env, run your server as usual ...
... if you already had such a database
vxdb02$ dropdb ass1
... create a new empty atabase
vxdb02$ createdb ass1
... load the database, saving the output in a file called log
vxdb02$ psql ass1 -f ass1.dump
... examine the database contents
vxdb02$ psql ass1
```

Database loading should take less than 1 second on `vxdb02`, and will produce messages like:

```
CREATE DOMAIN
CREATE TYPE
SET
CREATE TABLE
COPY number
ALTER TABLE
```

The `ass1.dump` file contains the schema and data in a single file, along with a simple PLpgSQL function (`dbpop()`), which counts the number of tuples in each table.

If you're running PostgreSQL at home you can download the dump file here: `ass1.dump`.

Think of some questions you could ask on the database (e.g. like the ones in the lectures) and work out SQL queries to answer them.

One useful query is

```
ass1=# select * from dbpop();
```

This will give you a list of tables and the number of tuples in each. The `dbpop()` function is written in PLpgSQL, and makes use of the PostgreSQL catalog. We'll look at this later in the term.

Your Tasks

Answer each of the following questions by typing SQL or PLpgSQL into the `ass1.sql` file. You may find it convenient to work on each question in a temporary file, so that you don't have to keep loading all of the other views and functions each time you change the one you're working on. Note that you can add as many auxuiliary views and functions to `ass1.sql` as you want. These views and functions can be used in answering as many questions as you want. However, their definition must appear in the `ass1.sql` file before their first usage. Also, make sure that everything that's required to make all of your views and functions work is in the `ass1.sql` file before you submit.

Note #1: many of the queries are phrased in the singular e.g. "Find the cheese that ...". Despite the use of "cheese" (singular), it is possible that multiple cheeses satisfy the query. If this is the case you should return all cheeses that satisfy the query. Because of this you should not use `LIMIT 1`.

Note #2: the database is nowhere near a complete picture of cheese in the Real World. Treat each question as being prefaced by "According to the cheese database database ...".

Note #3: you can assume that the names for cheese styles are unique; you *cannot* assume this for cheese names or the names of cheese makers.

Note #4: do not worry about the order of tuples in your result; we will apply `order by` when testing the queries (see the ordering in the Examples page).

There are examples of the results of each view and function in the Examples page.

Q0 (2 marks)

Style mark

Given that you've already taken multiple programming courses, we should be able to assume that you'll express your code with good style conventions. But, just in case ...

You must ensure that your SQL queries follow a consistent style. The one I've been using in the lectures is fine. An alternative, where the word `join` comes at the start of the line, is also OK. The main thing is to choose one style and use it consistently.

Similarly, PLpgSQL should be laid out like you would lay out any procedural programming language. E.g. bodies of loops should be indented from the `for` or `while` statement that introduces them. E.g. the body of a function should be indented from the `begin`...`end`.

Ugly, inconsistent layout of SQL queries and PLpgSQL functions will be penalised.

Q1 (1 mark)

Write an SQL view `Q1(maker,founded)` that returns the name of the oldest cheese maker and the year they were founded.

Q2 (1 mark)

Some cheesemakers get lazy, don't think up creative names for their cheeses, and simply name them after the style (e.g. a cheddar called "Cheddar"). Write an SQL view `Q2(cheese,maker)` that returns a list of cheeses whose name is exactly the same as their style. (Must match exactly, including case and punctuation).

Q3 (1 mark)

Write an SQL view `Q3(cheese,maker)` that returns a list of cheeses which are described as "crumbly" (in their style notes).

Q4 (1 mark)

Write an SQL view `Q4(hardness,ncheeses)` that gives a list of hardness levels (soft,semi-soft,etc.) and how many cheeses are produced with each hardness level. Order the list by hardness level.

Q5 (1 mark)

Write an SQL view `Q5(style)` that returns the most popular (i.e. most frequently made) cheese style.

Q6 (2 marks)

Write an SQL view `Q6(country)` that returns the name of the country that makes the most distinct styles of cheese.

Q7 (2 marks)

Write an SQL view `Q7(cheese,maker,aged,min_aging,max_aging)` which gives a list of cheeses which are aged outside the normal range of aging for cheeses of their style (i.e aged for less than the minimum aging time or more than the maximum aging time).

Q8 (3 marks)

Write a PLpgSQL function `Q8(partial_name text)`, that takes part of the name of a cheese maker and returns a set of tuples for all cheese makers that match the partial name (case insensitive matching). Cheesemakers must appear in alphabetical order on their names. Each matching tuple contains the full name of the cheese maker and their location, in the order town,region,country. Only non-null components of the location should be included, and should be comma-separated, with one space after the comma.

Q9 (4 marks)

Write a PLpgSQL function `Q9(partial_name text)`, that takes part of the name of a cheese maker and returns a set of tuples for all cheese makers that match the partial name (case insensitive matching). Cheesemakers must appear in alphabetical order on their names. For each cheese maker, give a list of tuples for all of the cheeses that the maker produces. Each tuple should have three fields: the maker, the name of the cheese and its style. Only the first tuple in the set for a given cheese maker, should have the cheese maker's name; this field should be null for all subsequent cheeses by that cheese maker. Cheeses for a given cheese maker should be listed in alphabetical order of their name.

Submission and Testing

We will test your submission as follows:

- create a testing subdirectory
- create a new database *`TestingDB`* and initialise it with `ass1.dump`
- load your work via the command: `psql TestingDB -f ass1.sql` (using your `ass1.sql`)
- run auto-marking on your views and functions
- drop the *`TestingDB`* database
- create a new database *`TestingDB`* and initialise it with an unseen dump file (same schema, different data)
- load your work via the command: `psql TestingDB -f ass1.sql` (using your `ass1.sql`)
- run auto-marking on your views and functions

Note that there is a time-limit on the execution of queries. If any query takes longer than 3 seconds to run (you can check this using the **\timing** flag) your mark for that query will be reduced.

Your submitted code must be *complete* so that when we do the above, your `ass1.sql` will load without errors. If your code does not work when installed for testing as described above and the reason for the failure is that your `ass1.sql` did not contain all of the required definitions, you will be penalised by a 2 mark penalty. If you code does not load because your definitions are in the wrong order, there will be a 1 mark penalty.

Before you submit, it would be useful to test whether the files you submit will work on `vxdb02`, by doing the following:

```
vxdb02$ dropdb ass1
vxdb02$ createdb ass1
vxdb02$ psql ass1 -f ass1.dump
vxdb02$ psql ass1 -f ass1.sql
```

These commands may produce information messages, but they *should not* produce any errors.

A Clear Understanding of Responsibilities

Our Commitment to You

We value transparency and strive to ensure a clear understanding of our responsibilities and expectations. In this regard, we would like to outline some of our responsibilities to you:

Assignment Release and Marking

- We will release assignments on time and ensure they are as error-free as possible.
- Your assignments will be marked within two weeks
- If any updates or minor corrections are required, we will update the specification and clearly communicate these changes.

Addressing Errors and Providing Support

- In the event of a significant error that impacts the assignment, we will extend the deadline to accommodate the time constraints our students may face. If this happens, we will communicate it clearly.
- We are committed to answering your questions, whether in person or online. For online inquiries, we will make sure to respond within two business days.
- Regular consultation hours and extra weekly help sessions will be conducted to address any questions and provide assistance.

Your Responsibilities

To ensure a fair and productive learning experience, we kindly request that you:

- Read and fully understand the assignment requirements.
- Complete the assignment without engaging in any form of cheating or plagiarism. For detailed information on academic integrity, please refer to the following link: https://www.student.unsw.edu.au/plagiarism/integrity
- If you face any challenges, seek assistance through the provided channels, such as consultations, help sessions or the forum.
- Allow up to two business days for your online questions to be answered.
- Regularly check the specification page and notice board for any updates or changes.
- Perform comprehensive testing and checking of your solutions.
- Test your solutions on the CSE server before submitting.

Have fun, *jas*